

JOB PORTAL MANAGEMENT SYSTEM

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

B ANJIKESAVA (VTU25502)

10211CS224

Full Stack Application Development

Slot : E1



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled “Job Portal Management System” by B ANJIKESAVA (VTU25502) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr.Hemamalini.S

Assistant Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Co-ordinator

Dr. Sumithra M

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(B ANJIKESAVA)

Date: 06-05-2026

ABSTRACT

The project presents the design and development of a modern Job Portal Management System aimed at bridging the gap between job seekers and employers. Built with a robust Spring Boot RESTful backend, a MySQL database, and a dynamic React frontend featuring a bold Neo-Brutalist design, the platform offers a highly responsive and engaging user experience. The system incorporates role-based access control, catering to distinct personas: Students, Employers, and Administrators. Key features include an intuitive job search and application tracking mechanism for students, alongside an advanced Employer Dashboard equipped with applicant management and an AI-driven interview question generator. To ensure security and system integrity, the application implements real-time SMTP-based OTP email verification and JWT authentication. By streamlining the recruitment workflow and providing advanced filtering and communication tools, this portal serves as a comprehensive, secure, and user-centric solution for modern employment networking.

Keywords: Job Portal, Full-Stack Development, React, Spring Boot, Role-Based Access Control, Recruitment Automation, Neo-Brutalist UI, Secure Authentication

LIST OF FIGURES

1.1	Framework and Architecture	3
3.1	Frontend And Backend Architecture	11
4.1	System Architecture Diagram	13
4.2	DataFlow Diagram	14
5.1	Login page for Employer and Student Authentication	17
5.2	Student Dashboard	18
5.3	Admin/Employer Dashboard	19

LIST OF TABLES

2.1 Feature Comparison with Market Leaders 6

7.1 SDG Mapping for the Project 23

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
ATS	Applicant Tracking System
CI	Continuous Integration
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JWT	JSON Web Token
ORM	Object Relational Mapping
OTP	One-Time Password
RBAC	Role-Based Access Control
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
UI	User Interface
UX	User Experience

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	2
1.3 Scope of the Project	2
1.4 Framework	3
2 SURVEY OF SIMILAR APPLICATION IN MARKET	4
2.1 Global Job Portals	4
2.2 Company Insight Platforms	4
2.3 Indian Job Portals	4
2.4 AI-Driven Recruitment Platforms	5
2.5 Identified Research Gap	5
2.6 Comparison of Existing Systems	5

2.7	Need for Proposed System	5
3	FRAMEWORK DESCRIPTION	7
3.1	Frontend Implementation	7
3.1.1	Environment Setup	7
3.1.2	Structure of the Project	7
3.1.3	Execution Procedure	9
3.2	Backend Implementation	9
3.2.1	Environment Setup	9
3.2.2	Structure of the Project	9
3.2.3	Execution Procedure	10
3.3	Database Implementation	11
3.3.1	Database Configuration	11
3.3.2	Schema Structure	12
3.3.3	Tables Created	12
4	METHODOLOGIES	13
4.1	Project Architecture	13
4.2	DataFlow Diagram	14
4.3	Module 1	14
4.4	Module 2	15
4.5	Module 3	15
5	RESULTS AND DISCUSSIONS	17
6	CONCLUSION AND FUTURE ENHANCEMENTS	20
6.1	Conclusion	20
6.2	Future Enhancements	20

7 ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG 21

7.1 Mapping of the Project to Complex Engineering Problem (CEP) . . 21

7.2 Mapping of the Project to Complex Engineering Activities (CEA) . 22

7.3 SDG Mapping (CEP Section) 22

8 SOURCE CODE 24

8.1 Source Code 24

References 27

Chapter 1

INTRODUCTION

1.1 Introduction

The rapid evolution of the digital landscape has fundamentally revolutionized the global recruitment process, transitioning it from static, fragmented methods to dynamic, data-driven ecosystems. Traditional approaches to job hunting and talent acquisition are often hindered by time-consuming manual screenings, lack of real-time communication, and administrative bottlenecks that frustrate both recruiters and job seekers. To address these challenges, the Job Portal Management System is designed as a centralized, high-performance platform that seamlessly aligns the needs of candidates with modern employers. By integrating a robust Spring Boot backend and a high-performance React frontend with a Neo-Brutalist design, the system ensures a secure, scalable, and frictionless experience. It also incorporates advanced features such as automated application tracking, AI-driven question generation, role-based access control, and real-time notifications to optimize the hiring lifecycle and enhance efficiency.

1.2 Aim of the Project

The primary aim of this project is to design and develop a secure, full-stack Job Portal Management System that facilitates seamless interaction between job seekers and recruiters. It focuses on providing a high-performance environment with role-based dashboards, real-time security verification (SMTP OTP), and AI-driven productivity tools to simplify the modern hiring process.

1.3 Scope of the Project

The scope of this project encompasses several critical functional areas that collectively enhance the recruitment process. It includes secure onboarding through a robust authentication system that leverages JWT and real-time SMTP-based email OTP verification to ensure user identity and data protection. The system provides centralized job management, enabling employers to efficiently create, update, and oversee job listings while also accessing applicant statistics through a dedicated dashboard. For job seekers, it facilitates seamless job discovery and tracking, allowing students to search, filter, and apply for jobs while monitoring their application status in real time. Additionally, the platform incorporates automated recruitment assistance by integrating an AI-powered tool that generates role-specific interview questions for employers. Furthermore, it features an Applicant Tracking System (ATS), offering a specialized interface for employers to manage candidate profiles, review resumes, and update application statuses such as shortlisting or rejection, thereby streamlining the entire hiring workflow.

1.4 Framework

The project is developed using a three-tier architecture to ensure scalability, security, and clear separation of concerns. The presentation layer (frontend) is built using React.js with Vite, styled using custom Vanilla CSS following a Neo-Brutalist design approach, and utilizes React Router DOM for seamless navigation. The application or logic layer (backend) is implemented using Java Spring Boot, incorporating Spring Security and JWT for secure authentication and authorization, and exposes functionality through RESTful web services. The data layer is managed using a MySQL database, with Spring Data JPA (Hibernate) serving as the ORM to efficiently handle database interactions and relationships.

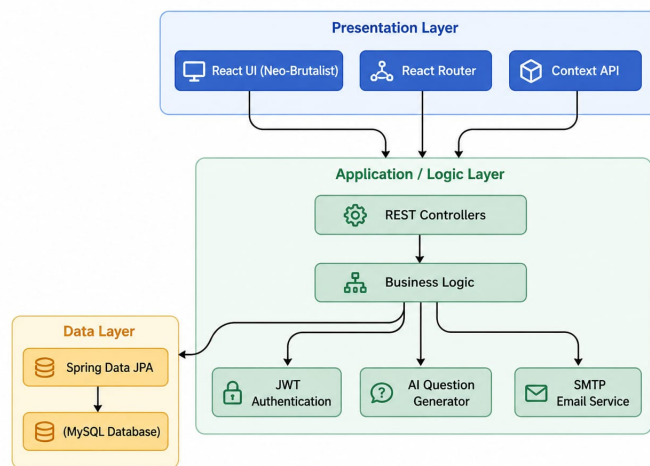


Figure 1.1: Framework and Architecture

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

2.1 Global Job Portals

Leading platforms like LinkedIn and Indeed dominate the global recruitment space by offering extensive job listings, professional networking, and simplified application processes. These platforms focus on connecting recruiters and candidates at scale with user-friendly interfaces and wide reach.

2.2 Company Insight Platforms

Glassdoor stands out by providing detailed insights into company culture, salary structures, and interview experiences. This helps candidates make informed career decisions beyond just applying for jobs.

2.3 Indian Job Portals

In India, platforms such as Naukri.com and Internshala are widely used. They cater to both experienced professionals and fresh graduates, offering targeted job

listings, internships, and skill-based opportunities.

2.4 AI-Driven Recruitment Platforms

Modern platforms like Monster and ZipRecruiter integrate advanced features such as resume parsing and AI-driven job recommendations to improve hiring efficiency and candidate matching.

2.5 Identified Research Gap

Despite these advancements, most existing systems focus on specific aspects such as job listings, networking, or analytics individually. There remains a gap in delivering a unified platform that integrates secure authentication, AI-powered recruitment assistance, modern UI design, and a comprehensive applicant tracking system—an area this project aims to address.

2.6 Comparison of Existing Systems

To better understand the competitive landscape, the following table compares established market leaders with our proposed Job Portal Management System based on key technical and functional parameters.

2.7 Need for Proposed System

The necessity for the proposed Job Portal Management System arises from several critical limitations in current market offerings:

- **Cognitive UX Improvement:** Most current portals utilize cluttered, advertisement-heavy interfaces that increase user fatigue. Our system employs a bold Neo-

Feature	LinkedIn	Naukri.com	Internshala	Proposed System
Job Application	High	High	High	High
AI Interview/Question Bot	No	No	No	Yes
Neo-Brutalist UI Design	No	No	No	Yes
Integrated Support Chatbot	Limited	No	No	Yes
SMTP Email Verification	Yes	Yes	Yes	Yes
Real-time Analytics	Paid	Basic	Basic	Built-in

Table 2.1: Feature Comparison with Market Leaders

Brutalist design that prioritizes readability and essential navigation.

- **AI-Assisted Screening:** While traditional portals act as passive bulletin boards, our system introduces an active AI Question Generator and Interview Bot, allowing employers to screen candidates more effectively and helping students prepare for roles.
- **Streamlined Communication:** There is a significant need for real-time engagement. By integrating a persistent Chatbot widget and automated SMTP-based notifications, we ensure that no candidate or employer is left in the “dark” regarding application status.
- **High-Security Standards:** By leveraging JWT-based stateless authentication and secure file handling for resumes, the proposed system provides a safer environment for sensitive professional data compared to many generic platforms.
- **Unified Role Management:** The system bridges the gap by providing deeply specialized dashboards for both Employers and Students within a single, cohesive architecture, eliminating the need for third-party tracking tools.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend of the Job Portal Management System is developed using a modern JavaScript-based technology stack to ensure high performance and responsiveness. It is built using React.js along with Vite as the development and build tool. React is used to create dynamic and reusable user interface components, while Vite enables fast development through efficient Hot Module Replacement. Node.js and npm are used as the runtime environment and package manager to manage dependencies. Additional libraries such as Axios are used for handling API requests, Framer Motion is used to implement smooth animations, and Lucide-React provides clean and scalable icons. The development is carried out using Visual Studio Code, which ensures code quality with tools like ESLint and Prettier. The frontend is executed in a web browser and communicates with the backend through REST APIs.

3.1.2 Structure of the Project

- The frontend follows a **component-based architecture** to enhance modularity and maintainability.

- The project is organized into different directories within the **src folder**.
- The **components folder** contains reusable UI elements such as:
 - Navigation bar
 - Chatbot widget
 - Buttons
- The **pages folder** includes major views such as:
 - Landing page
 - Login page
 - Student dashboard
 - Employer dashboard
- Static resources like images, fonts, and stylesheets are stored in the **assets folder**.
- The **services folder** manages API communication with the backend using Axios.
- The main entry files include:
 - **App.jsx** – Handles routing
 - **main.jsx** – Manages application initialization and global state
- **React Router** is used for navigation within the application.
- This structured approach ensures:
 - Scalability
 - Maintainability
 - Ease of development

3.1.3 Execution Procedure

To execute the frontend application, open the terminal and navigate to the frontend project directory. Install all required dependencies using the command `npm install`. After successful installation, run the application using `npm run dev`. This starts the development server provided by Vite. The application can then be accessed through a web browser at `http://localhost:5173` or the specified port. The frontend will interact with the backend server to fetch and display dynamic data.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend of the Job Portal Management System is developed using Java and the Spring Boot framework to ensure scalability and security. Java Development Kit (JDK) 17 is used as the programming environment. Spring Boot simplifies the development of RESTful web services and provides built-in features for dependency injection and configuration. Maven is used for project build and dependency management. MySQL is used as the relational database to store user data, job postings, and application details. Postman is used for testing API endpoints during development. The backend is designed to handle authentication, authorization, and data processing efficiently.

3.2.2 Structure of the Project

- The backend follows a **layered architecture** to maintain separation of concerns.
- The **controller package** handles incoming HTTP requests and maps them to appropriate services.

- The **service layer** contains the business logic and processes application data.
- The **model package** defines entity classes that correspond to database tables.
- The **repository layer** provides database access using JPA repositories for CRUD operations.
- The **security package** implements authentication and authorization using Spring Security and JWT tokens.
- The **exception package** manages error handling and ensures consistent API responses.
- This architecture improves:
 - Code organization
 - Maintainability
 - Scalability

3.2.3 Execution Procedure

To run the backend application, ensure that the MySQL server is running and the required database is created. Configure the `application.properties` file with the correct database URL, username, and password. Build the project using the command `mvnw clean install`. After building, run the application using `mvn spring-boot:run` or by executing the main class. The backend server will start on port 8081 and will be ready to handle requests from the frontend.

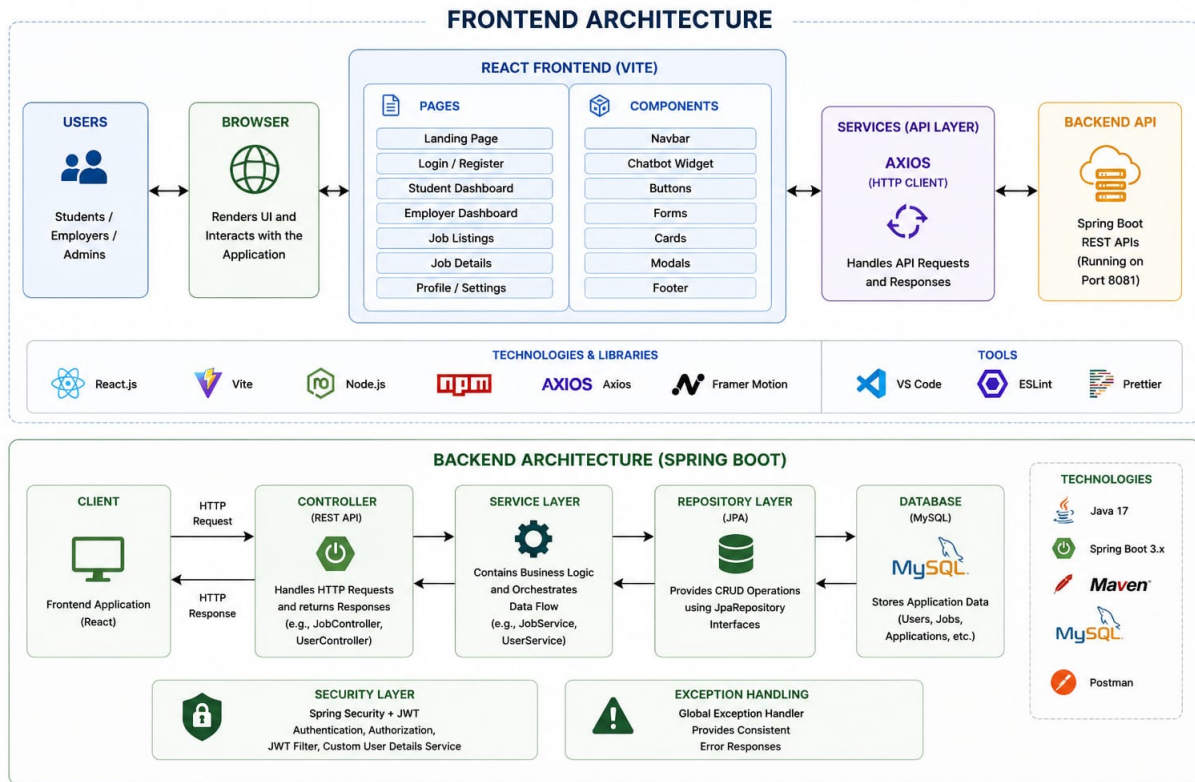


Figure 3.1: Frontend And Backend Architecture

3.3 Database Implementation

3.3.1 Database Configuration

The Job Portal Management System utilizes MySQL as its primary relational database management system (RDBMS). Connectivity between the Java backend and the database is established using Spring Data JPA and Hibernate ORM, with configuration parameters defined in the application.properties file. This setup ensures robust data persistence, secure connection pooling, and efficient transaction management. The database is responsible for storing all critical application data, including user credentials, job post details, application statuses, and resume metadata.

3.3.2 Schema Structure

The database schema is designed using an Entity-Relationship (ER) model that ensures data integrity and minimizes redundancy. The core entities—User, Job, Application, and Resume—are linked through strategic relationships. Primary keys are used for unique identification, while foreign keys are implemented to maintain referential integrity across the platform. For example, a “Many-to-One” relationship exists between Applications and Users, ensuring that each application is correctly mapped to its respective candidate.

3.3.3 Tables Created

The following main tables have been implemented in the database to support the system’s core functionalities:

- **users:** Stores registration details, encrypted passwords, and roles (Student, Employer, or Admin) required for role-based access control.
- **jobs:** Contains details about various job openings, including titles, descriptions, requirements, and company information posted by employers.
- **applications:** Records the link between candidates and jobs, tracking the status of each submission (e.g., Pending, Shortlisted, Rejected).
- **resumes:** Manages metadata and file paths for uploaded candidate documents to facilitate efficient retrieval by recruiters.
- **otp_storage:** (Optional) Stores temporary security tokens for email verification and password resets to ensure platform security.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The system uses a layered architecture where users access the application through a web browser connected to a React.js frontend. The frontend communicates with a Spring Boot backend via REST APIs, where security, business logic, and data processing are handled. The backend interacts with a MySQL database for data storage and integrates external services like email notifications and AI features, ensuring a smooth and efficient workflow.

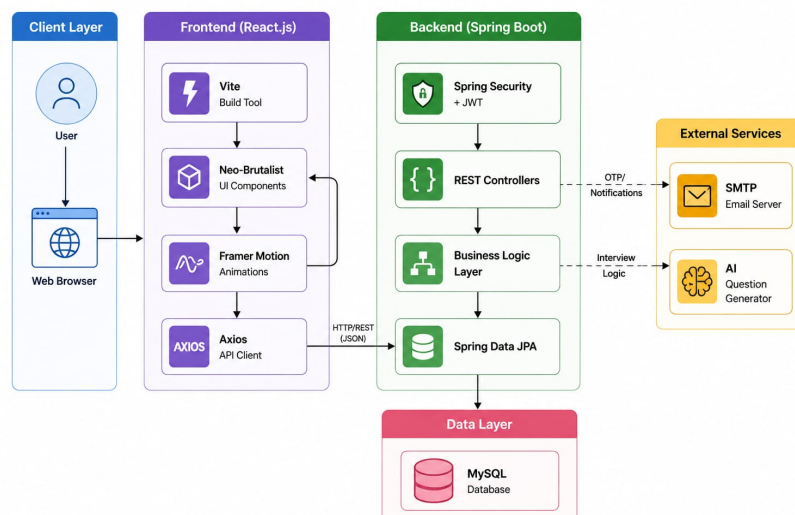


Figure 4.1: System Architecture Diagram

4.2 DataFlow Diagram

The DataFlow Diagram represents the overall interaction between external entities and the Job Portal Management System. It shows three primary users: Student, Employer, and Admin, all interacting with a central system. Students provide credentials, profiles, and job applications, and receive job listings, application status, and notifications. Employers interact by posting jobs and managing candidates, while receiving applicant data and analytics. Admins manage system configuration and users, and receive reports and logs. This diagram highlights the high-level data flow and system boundaries without showing internal processing details.

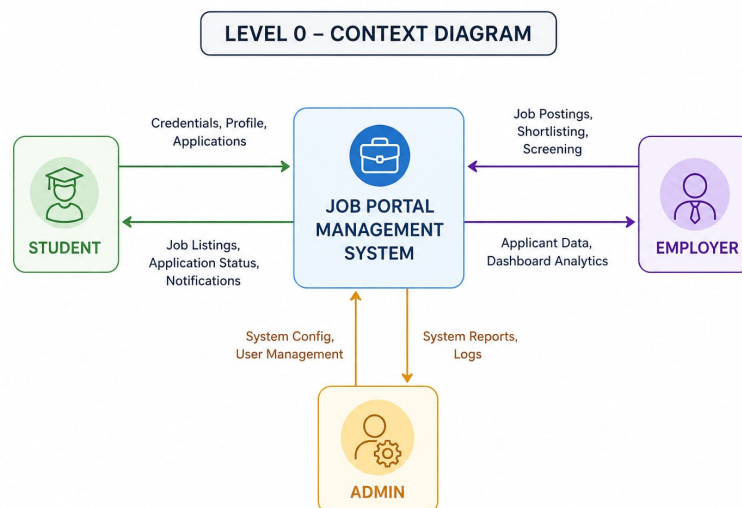


Figure 4.2: DataFlow Diagram

4.3 Module 1

User Management handles registration, login, and profile management with role-based access for students, employers, and admin. It includes secure authentication mechanisms such as password encryption and session handling. Users can update personal details, upload resumes or company information, and manage account settings. The module also supports role-based dashboards, ensuring that each user type

accesses only relevant features and data.

4.4 Module 2

Job Management includes job postings, applications, and candidate tracking between employers and students. Employers can create, edit, and delete job listings with detailed descriptions, required skills, and deadlines. Students can search and filter jobs based on criteria like location, skills, and experience level, and apply directly through the platform. The module also provides application status tracking, shortlist management, and interview scheduling features for streamlined recruitment.

4.5 Module 3

Notification and Analytics provides email notifications, application updates, and basic system insights. Users receive real-time alerts for job postings, application status changes, and important announcements. The analytics component offers insights such as number of applications, job popularity, and user activity trends. This helps employers make informed hiring decisions and allows administrators to monitor system performance and engagement.

Advantages of this Project

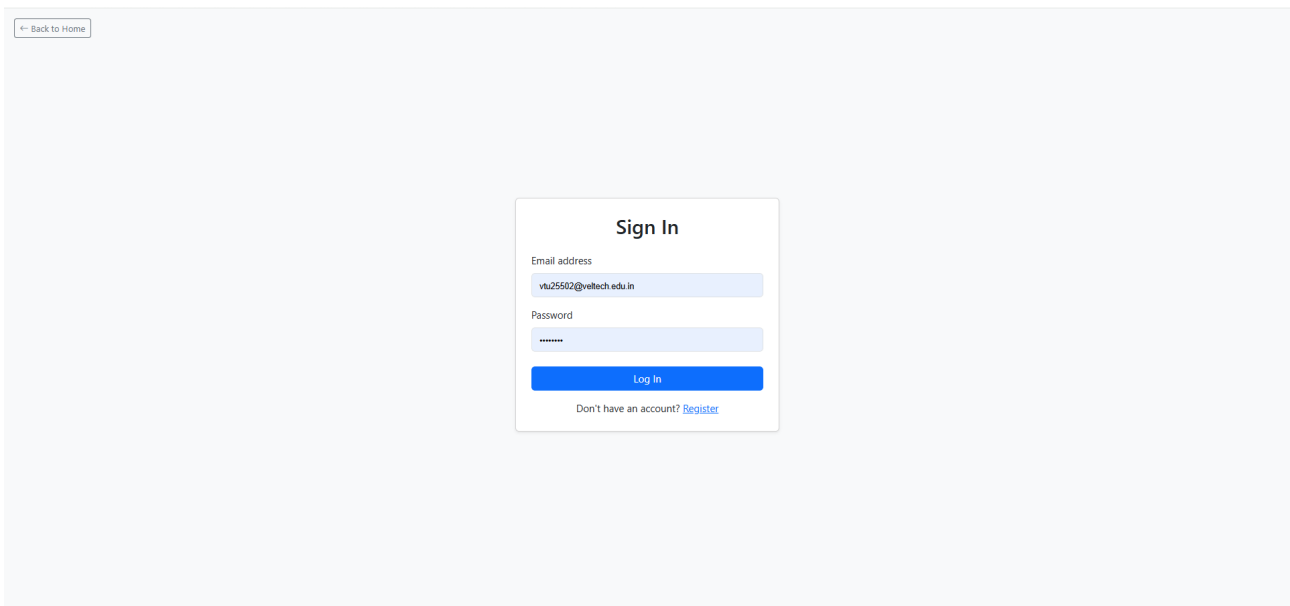
- a) Neo-Brutalist UX:** Provides a high-contrast, modern interface that improves readability and user engagement.
- b) AI-Powered Efficiency:** Automates screening using AI-based question generation and chatbot support.
- c) Secure Architecture:** Ensures safe data handling through JWT-based authentication and role-based access.

Disadvantages

- a) Internet Dependency:** Requires a stable internet connection for full functionality.
- b) Server Load:** AI features increase backend computational and infrastructure requirements.

Chapter 5

RESULTS AND DISCUSSIONS



The screenshot displays a web application's login interface. In the top-left corner, there is a small button labeled "← Back to Home". The main content area features a central "Sign In" form. This form includes two input fields: "Email address" with the value "vts25502@veltech.edu.in" and "Password" with masked characters "*****". Below these fields is a prominent blue "Log In" button. At the bottom of the form, a link reads "Don't have an account? [Register](#)". The entire form is set against a light gray background.

Figure 5.1: Login page for Employer and Student Authentication

The figure shows the login page where students (job applicants) and employers (job providers) can enter their credentials to access the system. Users can also log in via Google authentication (Firebase), and based on their roles, they are redirected to their respective dashboards.

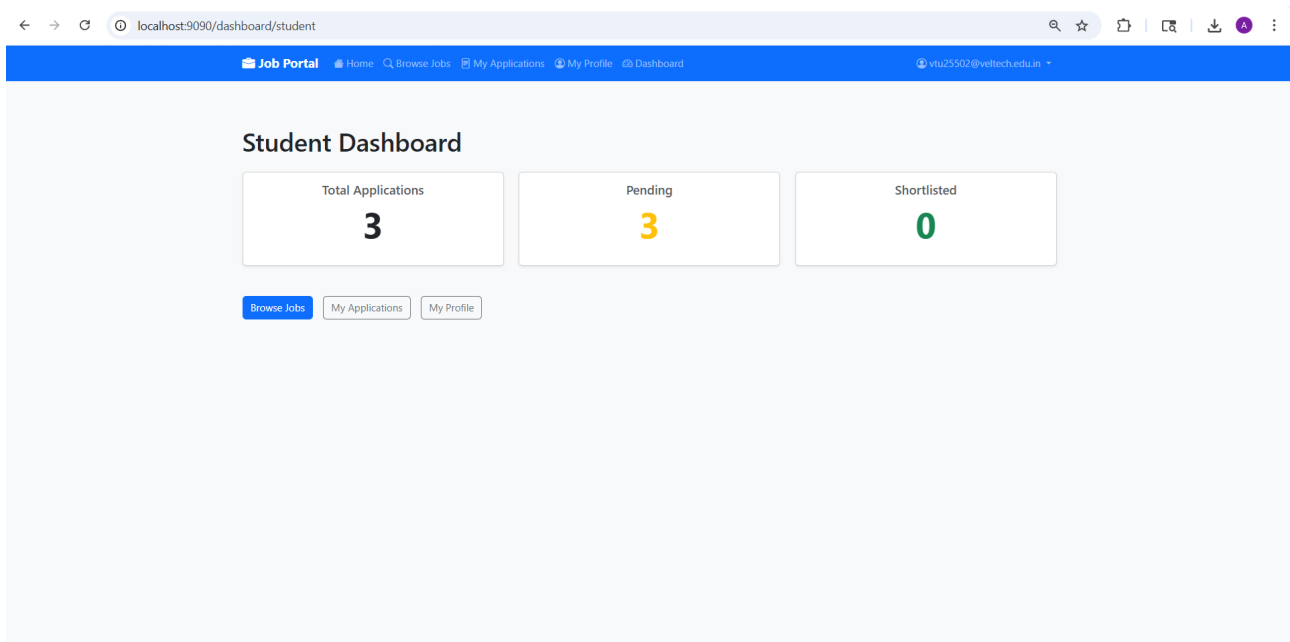


Figure 5.2: Student Dashboard

The student dashboard (shown above) provides a centralized interface for managing all job-related activities efficiently. It enables users to track job applications in real time with clear status indicators such as applied, under review, shortlisted, and rejected, helping them monitor their progress easily. Students can upload and manage resumes, browse and apply for jobs based on their preferences, and update their profiles to improve visibility to employers. The dashboard also offers features like personalized job recommendations, notification alerts for application updates, and access to interview preparation tools. Additionally, an integrated chatbot assists users by providing instant guidance, answering queries, and offering suggestions to enhance their job search and application process. This interactive and user-friendly design ensures a smooth and productive experience for job seekers.

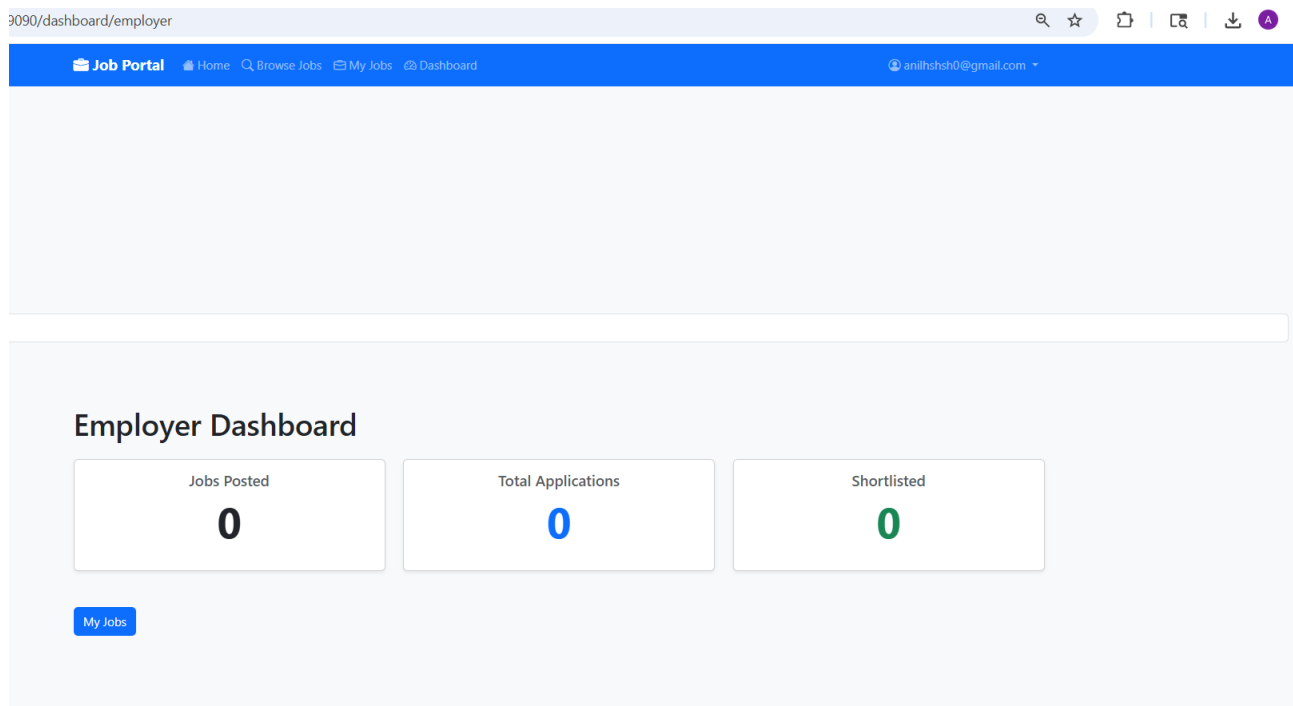


Figure 5.3: Admin/Employer Dashboard

The admin (employer) dashboard (shown above) provides a comprehensive overview of job postings and recruitment activities through an analytics-driven interface. It allows employers to create and manage job listings, monitor active and total job postings, and track the number of applications received. The dashboard presents key insights such as application trends over time, status distribution of candidates (under review, shortlisted, rejected), and performance metrics like average applications per job, response rate, and shortlist rate. Employers can also communicate with applicants, manage company profiles, and post new job openings directly from the interface. This centralized system enables efficient hiring decisions by offering real-time data visualization, streamlined candidate management, and improved recruitment workflow.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Job Portal Management System provides an efficient platform for connecting job seekers and employers through a modern, secure, and user-friendly interface. It simplifies the recruitment process with features like real-time tracking, role-based access, and AI support.

6.2 Future Enhancements

The system can be improved by adding features such as personalized job recommendations, mobile application support, advanced analytics, and video interview integration to enhance usability and scalability.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	Requires in-depth engineering knowledge	WK3, WK4, WK5, WK6	Requires knowledge of Spring Boot, Spring MVC, Spring Security, JPA, and database design
WP2	Conflicting Requirements	Technical and non-technical conflicts	WK4, WK5, WK6	Balances role-based access control with usability
WP3	Depth of Analysis	Analytical and Design thinking	WK3, WK4, WK5	Entity-relationship modeling, REST API design, file upload integration
WP4	Familiarity	Partially new problems	WK4, WK8	Modern challenges beyond traditional CRUD systems

Level	Attribute	Characteristics	WKs	Justification
WP5	Applicable Codes	Limited standard solutions	WK6	Custom design for resume storage and job filtering
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6, WK7	Handles job seekers, employers, and admins
WP7	Interdependence	System-level integration	WK3, WK5, WK6	Integrates frontend, backend, database, and security

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of diverse technologies	Spring Boot, MVC, JPA, MySQL, Security, JavaMailSender
EA2	Level of Interactions	Complex module interactions	Handles job listings, authentication, resume uploads
EA3	Innovation	Modern technologies	Dashboards, filtering, file upload, email notifications

EA No.	Attribute	Characteristics	Justification
EA4	Consequences	Impact on society	Improves employment access and transparency
EA5	Familiarity	Beyond standard practices	Combines authentication, tracking, and analytics

7.3 SDG Mapping (CEP Section)

SDG No.	SDG Title	Relevance to the Project
SDG 8	Decent Work and Economic Growth	Facilitates employment by connecting job seekers with employers, supporting fair recruitment processes and improving career opportunities
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in recruitment using Spring Boot REST APIs, dynamic filtering, and automated notification systems
SDG 10	Reduced Inequalities	Provides a standardised, accessible digital platform for all job seekers regardless of geography, reducing barriers to employment opportunities
SDG 16	Peace, Justice and Strong Institutions	Ensures data security and role-based access control using Spring Security and password hashing, maintaining integrity and fairness in the hiring process

Table 7.1: SDG Mapping for the Project

Chapter 8

SOURCE CODE

8.1 Source Code

```
1 package com.example.jobportal;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import jakarta.persistence.*;
6 import org.springframework.data.jpa.repository.JpaRepository;
7 import org.springframework.web.bind.annotation.*;
8 import org.springframework.beans.factory.annotation.Autowired;
9
10 import java.util.List;
11
12 @SpringBootApplication
13 public class JobPortalApplication {
14     public static void main(String[] args) {
15         SpringApplication.run(JobPortalApplication.class, args);
16     }
17 }
18
19 /* ===== ENTITY ===== */
20
21 @Entity
22 class Job {
23
24     @Id
```

```

25     @GeneratedValue(strategy = GenerationType.IDENTITY)
26     private Long id;
27
28     private String title;
29     private String description;
30     private String location;
31     private double salary;
32
33     // Getters & Setters
34     public Long getId() { return id; }
35     public void setId(Long id) { this.id = id; }
36
37     public String getTitle() { return title; }
38     public void setTitle(String title) { this.title = title; }
39
40     public String getDescription() { return description; }
41     public void setDescription(String description) { this.description =
42         description; }
43
44     public String getLocation() { return location; }
45     public void setLocation(String location) { this.location = location; }
46
47     public double getSalary() { return salary; }
48     public void setSalary(double salary) { this.salary = salary; }
49 }
50
51 /* ===== REPOSITORY ===== */
52
53 interface JobRepository extends JpaRepository<Job, Long> {
54     List<Job> findByLocation(String location);
55 }
56
57 /* ===== CONTROLLER ===== */
58
59 @RestController
60 @RequestMapping("/jobs")
61 class JobController {
62
63     @Autowired

```

```
63     private JobRepository repo;
64
65     // Add Job
66     @PostMapping
67     public Job addJob(@RequestBody Job job) {
68         return repo.save(job);
69     }
70
71     // Get All Jobs
72     @GetMapping
73     public List<Job> getJobs() {
74         return repo.findAll();
75     }
76
77     // Search by Location
78     @GetMapping("/search")
79     public List<Job> search(@RequestParam String location) {
80         return repo.findByLocation(location);
81     }
82 }
```

References

- [1] LinkedIn Job Portal. Available: <https://www.linkedin.com/jobs/>
- [2] Indeed Job Search. Available: <https://www.indeed.com/>
- [3] Glassdoor. Available: <https://www.glassdoor.com/>
- [4] Naukri.com. Available: <https://www.naukri.com/>
- [5] Monster Jobs. Available: <https://www.monster.com/>
- [6] VMware Inc., “Spring Boot Reference Documentation,” Available: <https://docs.spring.io/spring-boot/docs/current/reference/ht>
- [7] Meta Platforms Inc., “React – A JavaScript Library for Building User Interfaces,” Available: <https://react.dev/>
- [8] Oracle Corporation, “MySQL Reference Manual,” Available: <https://dev.mysql.com/doc/>
- [9] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, May 2015. Available: <https://www.rfc-editor.org/rfc/rfc7519>